

Análisis y Diseño de Software
Curso 2025-2026

Práctica 5: Colecciones, Genéricos, Lambdas y Patrones de Diseño

Inicio: A partir del 13 de Abril.

Duración: ~4 semanas.

Entrega: En Moodle, a las 23:55 del Viernes 8 de Mayo (todos los grupos)

Peso de la práctica: 30%

El objetivo de esta práctica es diseñar programas genéricos que usen colecciones avanzadas, sean capaces de adaptarse a tipos paramétricos, y empleen expresiones lambda y patrones de diseño de manera práctica.

En esta práctica diseñaremos e implementaremos una librería para construir [árboles de decisión](#)¹. Los árboles de decisión permiten tomar una decisión con respecto a unos datos de entrada, donde la estructura de la decisión se codifica en forma de árbol, y las ramas del árbol contienen condiciones sobre los datos de entrada.

Mediante el uso de genéricos, crearemos una librería altamente reutilizable para crear árboles de decisión adaptados a un tipo paramétrico. Adicionalmente, implementaremos mecanismos para el [aprendizaje de árboles de decisión](#)² a partir de datos. El objetivo de esta parte no es el desarrollo de algoritmos sofisticados de inferencia – que ya estudiarás en la asignatura de aprendizaje automático – sino el diseño de una librería extensible y flexible que permita incorporarlos, usando un paradigma de orientación a objetos.

¹ https://en.wikipedia.org/wiki/Decision_tree

² https://en.wikipedia.org/wiki/Decision_tree_learning

Apartado 1. Extrayendo datos: de objetos a *Datasets* (2.25 puntos)

En tareas de aprendizaje automático es habitual almacenar los datos en *datasets*: conjuntos de valores organizados en forma tabular, con columnas que tienen un nombre y un tipo. Estos dataset podrán ser usados para entrenar modelos, como los árboles de decisión, y para analizar los datos disponibles.

En este apartado, diseñarás una clase genérica *Dataset* para la creación de datasets, parametrizable por el tipo de objetos que deben almacenar. La clase *Dataset* debe extraer las características (*features*) de interés de una colección de objetos del tipo paramétrico. Por ejemplo, una clase *Person* puede incluir atributos como nombre, edad, peso, altura y género, pero quizá no estemos interesados en guardar cada uno de estos atributos en el dataset. Más aún, debemos diseñar un mecanismo para que la clase *Dataset* sea capaz de extraer de los objetos el valor de los atributos de interés. Para ello, diseña una interfaz genérica *Featurizer*, que se encargue – entre otras cosas – de devolver el nombre de las features de interés, así como el valor que tienen dichas features en un objeto del tipo paramétrico. Para poder funcionar, la clase *Dataset* necesitará un *Featurizer*, y la colección de objetos que conforman el dataset.

La clase *Dataset* podrá devolver las features del dataset por su nombre: estas serán colecciones que contendrán el valor del atributo correspondiente de cada uno de los objetos del dataset. Puedes modelar las features del dataset con una clase genérica *Feature*, que almacenará valores que se asumen comparables. La clase *Feature* debe ser compatible con las listas estándar de Java, e incluir facilidades para obtener el valor mínimo, máximo y un mapa con la distribución de frecuencia de los valores almacenados.

El siguiente listado muestra un ejemplo de uso de la librería y el resultado esperado. Nótese que un *Featurizer* (como *PersonFeaturizer*) puede no estar limitado a devolver valores de atributos, sino que puede realizar conversión de valores. En particular, en el ejemplo se transforma el atributo booleano de *Person* a MALE o FEMALE. *Dataset* además tendrá funcionalidades para eliminar duplicados (filas de valores que son iguales).

```
public static void main(String[] args) {
    Dataset<Person> dataSet = buildDataSet();
    System.out.println("dataset: "+dataSet);

    dataSet.removeDuplicates();
    System.out.println("dataset w/o duplicates: "+dataSet);

    Feature<Integer> ages = dataSet.feature("age");
    System.out.println("Ages: "+ages);
    Collections.sort(ages);
    System.out.println("Ages sorted: "+ages);
    System.out.println("Min age: "+ages.min());
    System.out.println("Gender distribution: "+dataSet.feature("gender").distribution()); // freq. of each value
}

public static Dataset<Person> buildDataSet() {
    Person people [] = { new Person("Pedro", 66, 75, 180, true), // name, age, weight, height, male?
                        new Person("Ana", 47, 54, 158, false),
                        new Person("Luis", 34, 75, 176, true),
                        new Person("Rosa", 47, 54, 158, false)
    };

    Dataset<Person> dataSet = new Dataset<>(new PersonFeaturizer()); // A Featurizer for Person objects
    dataSet.addAll(people);
    return dataSet;
}
```

Salida esperada:

```
dataset: {age=[66, 47, 34, 47], weight=[75.0, 54.0, 75.0, 54.0], gender=[MALE, FEMALE, MALE, FEMALE]}
dataset w/o duplicates: {age=[66, 47, 34], weight=[75.0, 54.0, 75.0], gender=[MALE, FEMALE, MALE]}
Ages: [66, 47, 34]
Ages sorted: [34, 47, 66]
Min age: 34
Gender distribution: {MALE=2, FEMALE=1}
```

Nota: Una clase más completa tendría facilidades para persistir el dataset en ficheros CSV, realizar operaciones (por ejemplo, normalización), desordenar, partir el dataset, o reconstruir los objetos a partir de un fichero CSV. No obstante, para la práctica basta esta prueba de concepto.

Apartado 2. Árboles de decisión (2.25 puntos)

Los árboles de decisión (https://en.wikipedia.org/wiki/Decision_tree) son estructuras en forma de árbol que permiten tomar una decisión en base a unos datos de entrada. Dicho de otro modo, permiten clasificar (asignar una etiqueta) a unos datos de entrada, que modelamos como un objeto. Cada nodo del árbol tendrá condiciones sobre las *features* del objeto, y los nodos hoja del árbol serán las etiquetas que el árbol asigna a los objetos de entrada en la clasificación.

Siguiendo esta idea, debes diseñar una clase genérica `DecisionTree`, con métodos para poder construir el árbol de manera fácil. Usa expresiones lambda para especificar las condiciones de salida de cada nodo. Debes tener un método `predict`, que ejecute el árbol de decisión, proporcionando la etiqueta resultado de la entrada. Debes tener dos versiones de `predict`, uno que funcione con `Datasets` compatibles, y otro con colecciones de objetos del tipo paramétrico del árbol.

A modo de ejemplo, el listado de más abajo muestra un ejemplo de uso de la clase `DecisionTree`, y el resultado esperado.

```
public static void main(String[] args) {
    Dataset<Person> dataSet = buildDataSet(); // like in previous listing
    DecisionTree<Person> dt = buildPersonDecisionTree();

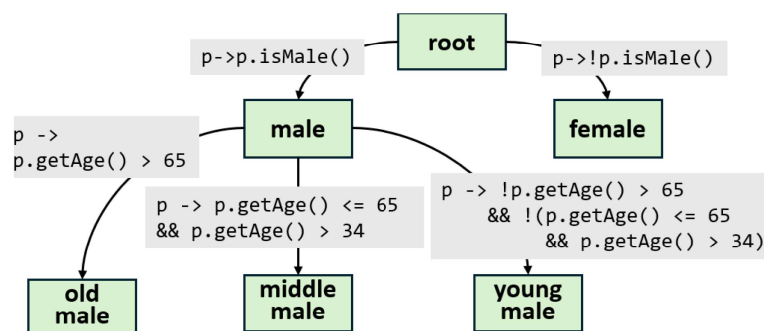
    System.out.println(dt.predict(dataSet));
    System.out.println(dt.predict(new Person("Miguel", 86, 72, 165, true), new Person("Clara", 42, 59, 162, false)));
}

public static DecisionTree<Person> buildPersonDecisionTree() {
    DecisionTree<Person> dt = new DecisionTree<>();
    dt.node("root") // nodo raíz, al ser el primero que se añade
        .withCondition("male", p -> p.isMale())
        .otherwise("female");
    dt.node("male") // como el nodo ya existe, se añaden condiciones sobre el
        .withCondition("old male", p -> p.getAge() > 65)
        .withCondition("middle male", p -> p.getAge() <= 65 && p.getAge() > 34)
        .otherwise("young male");
    return dt;
}
```

Salida esperada:

```
{old male=[Pedro(age: 66, male)], female=[Ana(age: 47, female), Rosa(age: 47, female)], young male=[Luis(age: 34, male)]}
{old male=[Miguel(age: 86, male)], female=[Clara(age: 42, female)]}
```

El árbol `dt` creado tendría la estructura de más abajo:



Nota: Si en la ejecución de `predict` sobre un objeto, ninguna de las condiciones de salida de un nodo se cumple (por ejemplo, porque no se añadió una opción “otherwise”), el objeto se queda en dicho nodo y no progresa. Debes diseñar un mecanismo para identificar esos casos y notificarlo.

Apartado 3. Generando predicados a partir de árboles (0,75 puntos)

Tal como los hemos diseñado, los árboles de decisión clasifican objetos, asignándoles una etiqueta. En este apartado extenderemos la clase `DecisionTree` para obtener explícitamente predicados (objetos `Predicate`) que indiquen si el objeto se puede clasificar con una etiqueta dada. Estos clasificadores se obtienen recorriendo el árbol en profundidad, hasta alcanzar el nodo con la etiqueta dada, haciendo la conjunción de todos los predicados encontrados en el camino. Esta extensión permitirá llamadas como: `Predicate<Person> isOldMale = dt.getPredicate("old male")`.

Apartado 4. Aprendiendo árboles (2.25 puntos)

En el apartado 2 has diseñado un API para la construcción de árboles de decisión, mediante llamadas a métodos. Con esa API, un programador puede diseñar árboles de decisión de manera explícita. En este apartado, diseñaremos clases que permitan aprender automáticamente árboles de decisión a partir de objetos (o Datasets) etiquetados. Esto es, crearemos una clase que, dado un dataset etiquetado (o una colección de objetos con un *etiquetador*), genere como salida un DecisionTree.

Un dataset etiquetado es un tipo especial de Dataset que además necesita un etiquetador. Podemos modelar etiquetadores mediante una interfaz genérica LabelProvider. Los LabelProvider permiten asignar una etiqueta a un objeto (compatible con el tipo paramétrico).

Existen numerosos algoritmos para el aprendizaje de árboles de decisión, pero optaremos por el enfoque más sencillo posible: los algoritmos *greedy*. Estos algoritmos van creando nodos y condiciones, eligiendo la “mejor feature” en cada paso. Se espera que el árbol generado, generalice: sea capaz de asignar etiquetas de manera razonable a objetos que no “ha visto” en el conjunto de objetos usado en el aprendizaje. Un esquema del algoritmo podría ser el siguiente.

```
function buildTree(data, availableFeatures)    // objetos/datos y lista de features disponibles
  if all labels are the same:                // todos los objetos en data se etiquetan igual
    return single node with that label

  feat := choose the best feature to split on // elegir la “mejor” feature -> de momento una aleatoria
  availableFeatures.remove(feat)             // se borra feat de la lista de features disponibles
  split data into subsets based on feat      // devolver tantos subconjuntos como valores distintos feat hay en data

  for-each subset do
    build subtree recursively                // añadir la condition “feat==value” y llamada recursiva con el subconjunto
                                          // de data { x in data | x.feat == value }
  return node with branches
```

De momento, la elección de la “mejor” feature puede ser aleatorio, ya que en el siguiente apartado usaremos distintas estrategias para su elección.

El listado de más abajo muestra un ejemplo de uso de las clases propuestas. Por un lado, LabeledDataset se crea con objetos Featurizer y LabelProvider (que en este caso asigna una etiqueta Boolean a objetos Weather). Por otro lado, el GreedyTreeLearner genera un DecisionTree que clasifica a objetos de tipo Weather. Nótese que las etiquetas que en principio eran de tipo Boolean en el LabeledDataset pasan a ser String en el DecisionTree.

```
public static DecisionTree<Weather> learnTree() {
    LabeledDataset<Weather, Boolean> dataSet = buildDataSet();
    GreedyTreeLearner<Weather, Boolean> learner = new GreedyTreeLearner<>();
    DecisionTree<Weather> tree = learner.learn(dataSet);
    return tree;
}

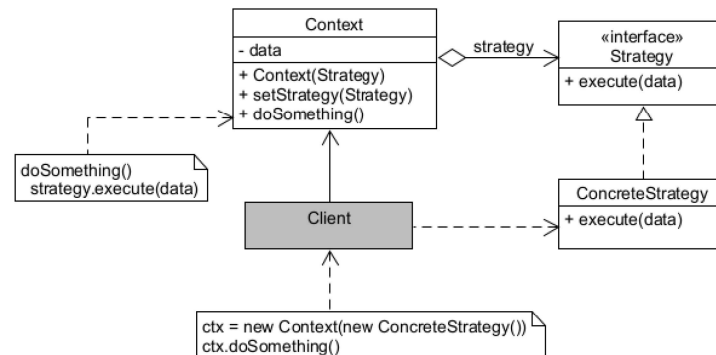
private static LabeledDataset<Weather, Boolean> buildDataSet() {
    Weather conditions [] = {
        new Weather(WeatherCondition.RAINY, Temperature.COLD),
        new Weather(WeatherCondition.RAINY, Temperature.HOT)
        // más objetos ...
    };
    LabeledDataset<Weather, Boolean> ds = new LabeledDataset<>(new WeatherFeaturizer(), new ShouldIPlayTennisToday());
    ds.addAll(conditions);
    return ds;
}
```

En el listado anterior, el LabeledDataset se crea con un featurizer (WeatherFeaturizer) y un etiquetador (ShouldIPlayTennisToday) que etiqueta los objetos Weather con un booleano (true si el día es cálido y soleado, y false en caso contrario). El método learn de la clase GreedyTreeLearner se encarga de devolver un DecisionTree, dado un LabeledDataset, y debe construirse otra versión del método que reciba un conjunto de objetos, un featurizer y un etiquetador.

Nota: El algoritmo greedy no es muy adecuado para trabajar con valores continuos, o features con muchos valores (como Integer). Puedes ignorar el problema, u opcionalmente mejorar el algoritmo para que permita comparaciones no limitadas a igualdad.

Apartado 5. Estrategias para la elección de la mejor *feature* (1.5 puntos)

La clase anterior GreedyTreeLearner puede parametrizarse con una estrategia para la elección de la mejor *feature* cuando se generan nuevos nodos. Puedes usar el patrón de diseño Strategy para hacer esta parametrización. Tienes un esquema del patrón más abajo.



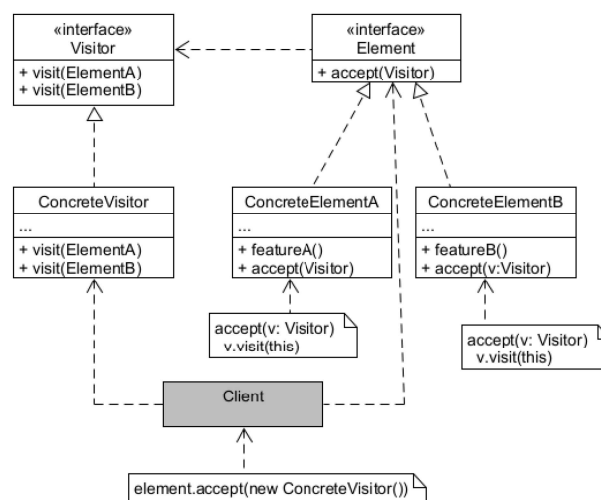
La idea del patrón es parametrizar un método (`Context.doSomething()`) mediante objetos compatibles con **Strategy**.

Crea al menos dos estrategias heurísticas de elección de la mejor *feature* a utilizar. Por ejemplo: puede ser una elección aleatoria, usando alguna métrica de clasificación errónea, el índice *Gini*, o la *Entropía* (https://en.wikipedia.org/wiki/Decision_tree_learning). Por ejemplo, una métrica de clasificación errónea puede calcularse con el pseudocódigo de más abajo:

```
for-each feature:
  Group data by feature value
  Collect the labels per group
  for-each group:
    Compute the majority label
    Count how many elements don't match it
  score = sum all those mismatches
return the feature with the lowest score
```

Apartado 6. Visitando el árbol: visualizadores (1 puntos)

Una funcionalidad útil para los árboles de decisión es poder visualizarlos. Podríamos añadir métodos para su visualización directamente en la clase **DecisionTree**. No obstante, estos métodos “ensuciarían” la clase, y deberíamos considerar métodos adicionales para visualizar el árbol en distintos formatos. Para poder añadir visualizadores de distinto tipo sin añadir métodos a las clases del árbol de decisión, usaremos el patrón de diseño **Visitor**. Tienes un esquema más abajo.



El patrón permite tener distintos **ConcreteVisitors**, que podrán visitar elementos de distinto tipo (**ElementA**, **ElementB**). Los elementos a visitar son conformes a una misma interfaz (**Element**), que tiene un método `accept` para aceptar un **Visitor**. Los elementos concretos implementarán dicho método, típicamente llamando a `visit`. Esta sobreescritura del método `accept` con exactamente el mismo código es necesario para poder realizar un “*doble dispatch*” (https://en.wikipedia.org/wiki/Double_dispatch):

por un lado, la ligadura dinámica al llamar a `element.accept` selecciona la clase `ConcreteElementA` o `ConcreteElementB`; por otro, la llamada de vuelta a `visit` selecciona el método apropiado de `ConcreteVisitor` dado el tipo del objeto que invoca (nótese que `visit` está sobrecargado).

Aplica este patrón en el contexto de la práctica para diseñar una solución extensible para la visualización de árboles de decisión. Implementa dos visualizadores: uno que genere texto para la visualización con *Graphviz* (<https://graphviz.org/>), y otro que genere texto plano con indentación.

Nota: Graphviz utiliza el formato DOT, con un esquema muy sencillo. Puedes usar ese formato on-line para probar las visualizaciones que generas aquí: <https://dreampuf.github.io/GraphvizOnline/>

Comentarios adicionales

- Además del correcto funcionamiento de la práctica, un aspecto fundamental en la evaluación será la **calidad del diseño**. Tu diseño debe utilizar los principios de orientación a objetos, además de ser claro, fácil de entender, extensible y flexible. Presta atención a la calidad del código, evitando redundancias.
- Organiza el código en **paquetes**.
- Crea **programas de prueba** para ejercitar el código **de cada apartado**, y entrégalos con tu código. Puedes usar programas de prueba con un `main`, o bien usar JUnit.

Normas de entrega

Se deberá entregar:

- Un directorio **src** con el código Java de todos los apartados, incluidos los datos de prueba y **testers adicionales** que hayas desarrollado en los apartados que lo requieren (puedes usar JUnit).
- Un directorio **doc** con la **documentación** generada.
- Una **memoria** en formato **PDF** con una pequeña descripción de las decisiones del diseño adoptadas para cada apartado, y **con el diagrama de clases** de tu diseño.

Se debe entregar un único fichero ZIP con todo lo solicitado, que deberá llamarse de la siguiente manera: GR<numero_grupo>_<nombre_estudiantes>.zip. Por ejemplo, Marisa y Pedro, del grupo 2213, entregarían el fichero GR2213_MarisaPedro.zip, de manera que cuando se extraiga haya una carpeta con el mismo nombre que contenga el directorio `src/`, el directorio `doc/`, y el PDF de la memoria.